

UNIVERSIDADE DE SÃO PAULO

Escola de Artes, Ciências e Humanidades — EACH

Programa de Pós-Graduação em Sistemas de Informação — Aluno Especial

SIN5032 — Experimentação em Aprendizado de Máquina Supervisionado

Primeiro Exercício Prático (TP1):

Planejamento de Estudo Exploratório para Detecção de Malware Android com Aprendizado de Máquina Supervisionado

Autor: Fernando Pinéo de Abreu

N.º USP: 17989742

Docente: Prof. Dr. Norton Trevisan Roman

São Paulo, abril de 2026

Resumo

A crescente proliferação de aplicativos maliciosos na plataforma Android representa um desafio significativo para a segurança da informação, motivando o desenvolvimento de métodos automáticos e escaláveis de detecção. Este trabalho apresenta o planejamento de um estudo exploratório que investiga a aplicação de algoritmos de Aprendizado de Máquina Supervisionado para a classificação de aplicativos Android como maliciosos (malware) ou benignos (goodware), a partir de *features* estáticas extraídas de permissões declaradas e chamadas a APIs do sistema operacional. O *dataset* utilizado é o TUANDROMD (*Tezpur University Android Malware Dataset*), composto por 4.464 instâncias e 241 variáveis independentes binárias, disponibilizado publicamente no UCI Machine Learning Repository. A análise exploratória de uma subamostra estratificada de 10% (446 instâncias) revelou desbalanceamento significativo entre as classes — 79,8% malware e 20,2% goodware — e identificou *features* com elevado poder discriminativo, como a permissão `RECEIVE_BOOT_COMPLETED` — predominante em malware — e chamadas a APIs de rede cuja **ausência** caracteriza aplicativos maliciosos, padrões consistentes com a literatura de segurança Android. Os algoritmos candidatos para a fase de execução são Random Forest, Support Vector Machine, K-Nearest Neighbors e Árvore de Decisão, a serem comparados por meio de validação cruzada estratificada com 10 *folds* e teste estatístico de Wilcoxon pareado ($\alpha = 0,05$), tendo o *F1-score* como métrica principal. Espera-se identificar quais *features* estáticas e quais algoritmos oferecem melhor capacidade de distinção entre as classes, contribuindo para o desenvolvimento de sistemas mais eficazes de detecção de malware Android.

1 INTRODUÇÃO

O sistema operacional Android domina o mercado global de dispositivos móveis, tornando-se simultaneamente a plataforma mais visada por desenvolvedores de *software* malicioso. Segundo o portal AV-ATLAS, do instituto independente AV-TEST, o número de aplicativos Android infectados já ultrapassa 35 milhões de amostras catalogadas (AV-TEST, 2024). O relatório *Mobile Threat Landscape 2024*, publicado pela Kaspersky Lab no Mobile World Congress de 2025, registrou 33,3 milhões de ataques envolvendo malware, adware e *software* indesejado em dispositivos móveis ao longo de 2024, com destaque para o crescimento de 196% nos ataques de Trojans bancários em relação ao ano anterior (KASPERSKY, 2025). Entre as ameaças mais prevalentes encontram-se adware (35% das detecções), Trojans e RiskTools, com mais de 1,1 milhão de pacotes maliciosos de instalação detectados no período. Esses números evidenciam que a ameaça ao ecossistema Android não é pontual, mas estrutural e crescente, impondo desafios contínuos às soluções de segurança.

As abordagens tradicionais baseadas em assinaturas mostram-se insuficientes frente a técnicas de ofuscação e malware polimórfico. O Aprendizado de Máquina Supervisionado surge como alternativa promissora, permitindo que modelos aprendam padrões discriminativos diretamente dos dados, sem depender de regras pré-estabelecidas (ALPAYDIN, 2014).

1.1 Objetivo Principal

Este trabalho tem como objetivo planejar e executar um estudo exploratório para investigar a capacidade de algoritmos de Aprendizado de Máquina Supervisionado em distinguir aplicativos Android maliciosos (malware) de benignos (goodware), a partir de *features* estáticas extraídas de permissões declaradas e chamadas a APIs do sistema Android.

1.2 Questão de Pesquisa

Quais *features* estáticas de aplicativos Android — permissões e chamadas a APIs do sistema — apresentam maior poder discriminativo entre malware e goodware, e quais algoritmos de Aprendizado de Máquina Supervisionado são mais promissores para essa classificação? A população-alvo desta pesquisa são aplicativos Android distribuídos por meio de lojas oficiais e não oficiais.

2 MATERIAIS E MÉTODOS

2.1 Moldura de Amostragem e Limitações

A moldura de amostragem é composta pelos aplicativos Android catalogados pela Universidade de Tezpur (Índia), cujas *features* estáticas foram extraídas e compiladas no

TUANDROMD. Embora a população-alvo abranja todos os aplicativos Android existentes, a moldura está limitada a aplicativos coletados em período e contexto específicos, podendo não refletir ameaças mais recentes ou de outras regiões geográficas.

Adicionalmente, o protocolo de seleção dos aplicativos pelos autores do TUANDROMD não é descrito em detalhes suficientes para confirmar aleatoriedade estrita na composição da base. Trata-se, portanto, de uma amostragem por conveniência dentro da moldura definida pelos autores (SHATNAWI et al., 2022), em que aplicativos coletados de fontes específicas — lojas oficiais e repositórios de malware catalogados — podem não representar proporcionalmente todas as famílias de malware ou categorias de goodwill existentes na população-alvo. Essa característica é comum a bases de dados públicas de segurança computacional e constitui limitação reconhecida na literatura (ISAAC; TANIMU, 2025): as conclusões do presente estudo são condicionadas a essa moldura e não devem ser generalizadas sem cautela a cenários com distribuições de aplicativos substancialmente distintas. Para fins de reprodutibilidade e escrutínio científico da base, o *dataset* completo — incluindo os nomes originais de todas as 241 *features*, a distribuição das classes e os valores brutos das instâncias — está disponível publicamente no UCI Machine Learning Repository (SHATNAWI et al., 2022), e a inspeção programática aqui reportada é integralmente reprodutível a partir do código disponibilizado com este trabalho.

2.2 Caracterização da Base de Dados

O *dataset* utilizado é o TUANDROMD (*Tezpur University Android Malware Dataset*), composto originalmente por 4.465 instâncias: 241 variáveis independentes binárias (representando permissões e chamadas a APIs Android) e uma variável-alvo categórica (*Label*: malware / goodwill).

2.2.1 Qualidade dos Dados e Tratamento de Valores Ausentes

A inspeção de qualidade identificou 242 valores ausentes (NaN), distribuídos uniformemente — exatamente 1 por coluna, incluindo a variável-alvo *Label*. A investigação revelou que todos os 242 NaN pertencem a uma única instância, cujas 241 *features* e o próprio rótulo estão ausentes simultaneamente.

Para identificar qualquer padrão sistemático que pudesse introduzir viés, a distribuição da instância problemática foi comparada à distribuição das 4.464 instâncias válidas. Como a instância corrompida não possui rótulo, ela não pertence a nenhuma das classes (malware ou goodwill), sendo impossível verificar se sua ausência afeta proporcionalmente alguma delas. As 4.464 instâncias restantes mantêm a distribuição original de 79,8% malware e 20,2% goodwill — proporção idêntica à do *dataset* completo com 4.465 instâncias — confirmando que a remoção não introduz viés sistemático entre as classes. Por não ser imputável (ausência do rótulo a torna inutilizável para aprendizado supervisionado), a decisão metodológica adotada foi sua remoção, resultando em *dataset* limpo de 4.464

instâncias (redução de 0,02% — impacto negligenciável). Após a remoção, o *dataset* apresentou 0 valores ausentes, confirmado programaticamente. Todo o procedimento foi executado sobre o *dataset* completo antes da extração da subamostra exploratória, garantindo ausência de *data leakage*.

Durante a inspeção de qualidade, identificou-se ainda que alguns nomes de colunas do *dataset* apresentam ofuscação intencional aplicada pelos autores: substrings dos nomes originais das APIs Android foram substituídas por termos arbitrários (por exemplo, `getLastKnownLocation` aparece no CSV como `getLastKgoodwarewnLocation`), padrão idêntico ao reportado no *dataset* TUNADROMD — versão pré-processada do TUANDROMD (SHATNAWI et al., 2023¹) —, onde o mesmo campo aparece como `getLastK0wnLocation`. Essa ofuscação é consistente com estratégia de anonimização descrita pelos autores para dificultar a engenharia reversa das regras de detecção. O tratamento adotado foi o remapeamento programático dos nomes ofuscados para os nomes reais das APIs Android, realizado imediatamente após o carregamento do *dataset*, antes de qualquer outra etapa do *pipeline*.

2.2.2 Verificação de Registros Duplicados

A verificação programática sobre o *dataset* limpo de 4.464 instâncias identificou 3.802 registros com vetores de *features* idênticos a pelo menos outro registro, resultando em 662 instâncias com vetores únicos. A investigação revelou ainda 2 vetores com *labels* conflitantes — instâncias com padrão idêntico de *features* rotuladas de forma oposta (malware e goodware) —, o que constitui ruído inerente ao processo de coleta e rotulação do *dataset* (SHATNAWI et al., 2022).

Esse fenômeno é documentado na literatura: Souza et al. (2024) identificaram e removeram duplicatas do TUANDROMD como etapa de preparação dos dados. Contudo, a decisão metodológica adotada neste estudo foi **manter** os registros duplicados, pelos seguintes motivos: (i) com 241 variáveis binárias, colisões de vetores entre aplicativos distintos são esperadas e refletem a distribuição real da população — dois aplicativos diferentes podem compartilhar exatamente o mesmo conjunto de permissões e APIs sem serem o mesmo aplicativo; (ii) a remoção reduziria o *dataset* de 4.464 para 662 instâncias (perda de 85,2%), operando todos os algoritmos candidatos abaixo do limiar mínimo de estabilidade de performance documentado na literatura — Random Forest requer mediana de 3.404 instâncias e SVM entre 1.000 e 3.000 para convergência do AUC (SOUZA, T. et al., 2024) —, além de introduzir viés positivo nas estimativas de *k*-fold cross-validation com amostras reduzidas (VAROQUAUX et al., 2017); (iii) os 2 vetores conflitantes representam 0,04% do *dataset* e seu impacto sobre os classificadores é negligenciável. Essa limitação é reconhecida e suas implicações sobre a generalização dos resultados são discutidas na Seção 4.

¹<https://archive.ics.uci.edu/dataset/813/tunadromd>

2.2.3 Distribuição das Classes e Desbalanceamento

A distribuição da variável-alvo revela desbalanceamento significativo: 3.565 instâncias malware (79,8%) e 899 goodware (20,2%). Esse desbalanceamento é característico de *datasets* de detecção de malware e constitui limitação metodológica, pois modelos sem tratamento adequado tendem a favorecer a classe majoritária, inflando a acurácia sem refletir desempenho real na detecção de goodware — padrão documentado mesmo em abordagens avançadas como seleção dinâmica de classificadores (SOUZA, J. et al., 2025). Por isso, o *F1-score* foi escolhido como métrica principal.

2.2.4 Subamostra Exploratória

Para a fase exploratória, foi extraída subamostra de 10% do *dataset* limpo (446 instâncias) por amostragem aleatória estratificada (`stratify=Label`, `random_state=42`), preservando a proporção original: 356 instâncias malware (79,8%) e 90 goodware (20,2%). As 4.018 instâncias restantes ficam reservadas para treinamento e validação no TP2, prevenindo *data leakage*.

2.3 Variáveis e Fatores

O TUANDROMD é composto por 241 atributos independentes e uma variável-alvo (*Label*), totalizando 242 colunas. Todos os atributos independentes são variáveis binárias de nível nominal (0 = ausente, 1 = presente), organizados em duas categorias funcionais: permissões declaradas no `AndroidManifest.xml` e chamadas a APIs do sistema Android. A Tabela 1 sintetiza essa estrutura com exemplos representativos de cada categoria; a lista completa dos 241 atributos está disponível no repositório público UCI Machine Learning Repository (SHATNAWI et al., 2022).

Tabela 1: Categorias de atributos do TUANDROMD

Categoria	Qtd.	Tipo / Nível	Exemplos representativos
Permissões do <code>AndroidManifest.xml</code>	213	Binária / Nominal	RECEIVE_BOOT_COMPLETED, ACCESS_FINE_LOCATION, READ_PHONE_STATE
Chamadas a APIs Android	28	Binária / Nominal	Ljava/net/URL;->openConnection, DexClassLoader;->loadClass, Ljava/lang/System;->load
Variável-alvo (<i>Label</i>)	1	Categórica / Nominal	malware (codificado como 1) ou goodware (codificado como 0)

Nota: representação binária — 0 indica ausência e 1 indica presença do atributo no aplicativo analisado.

A escolha de representar as *features* como variáveis binárias — em vez de contagens de chamadas ou frequências relativas — é uma decisão metodológica dos autores do *dataset* (SHATNAWI et al., 2022), que optaram pela presença ou ausência como sinal discriminativo, descartando informação de intensidade de uso. Essa representação é adequada ao objetivo de classificação binária e amplamente adotada na literatura de detecção de malware Android estático (SAEED et al., 2026; ISAAC e TANIMU, 2025), mas introduz uma limitação: dois aplicativos que chamam a mesma API com frequências muito distintas recebem o mesmo valor 1, perdendo-se potencial discriminativo adicional.

Os fatores de interesse observados neste estudo são: Fator 1 — Padrão de uso de APIs de conectividade e rede (chamadas que estabelecem conexões externas, tipicamente associadas à exfiltração de dados); Fator 2 — Mecanismos de execução dinâmica e persistência (carregamento de classes e bibliotecas em tempo de execução, associados a malware polimórfico); Fator 3 — Permissões de acesso a dados sensíveis do usuário (localização, estado do telefone e armazenamento, típicas de *spyware*).

2.4 Algoritmos Candidatos

Com base na revisão da literatura e nas características do *dataset* (*features* binárias, classificação binária, $n = 4.464$), os seguintes algoritmos são candidatos iniciais: (i) Random Forest — robusto a ruído e *features* irrelevantes; (ii) Support Vector Machine (SVM) — eficaz em alta dimensionalidade; (iii) K-Nearest Neighbors (KNN) — sensível à distribuição local; (iv) Árvore de Decisão — interpretável e adequada para *features* binárias. A escolha final será definida após exploração da subamostra (Seção 3).

2.5 Divisão dos Dados e Validação

Para o TP2, os dados serão validados com $k = 10$ *folds* estratificados, preservando a proporção entre classes em cada *fold*. Métricas: F1-score (principal — robusto ao desbalanceamento), AUC-ROC e acurácia balanceada. A comparação entre algoritmos utilizará o teste de Wilcoxon pareado ($\alpha = 0,05$) sobre os F1-scores dos 10 *folds*.

2.6 Comunicação dos Resultados

O público-alvo são pesquisadores e profissionais de segurança computacional com conhecimento intermediário em Aprendizado de Máquina. Os resultados serão apresentados por meio de tabelas comparativas de desempenho, gráficos de distribuição de *features* e métricas com intervalos de confiança.

3 RESULTADOS PRELIMINARES

3.1 Distribuição das Classes na Subamostra

A subamostra de 446 instâncias (10% estratificado) preservou fielmente a distribuição original: 356 instâncias malware (79,8%) e 90 goodware (20,2%), confirmando a

eficácia da estratificação. A Figura 1 ilustra essa distribuição.

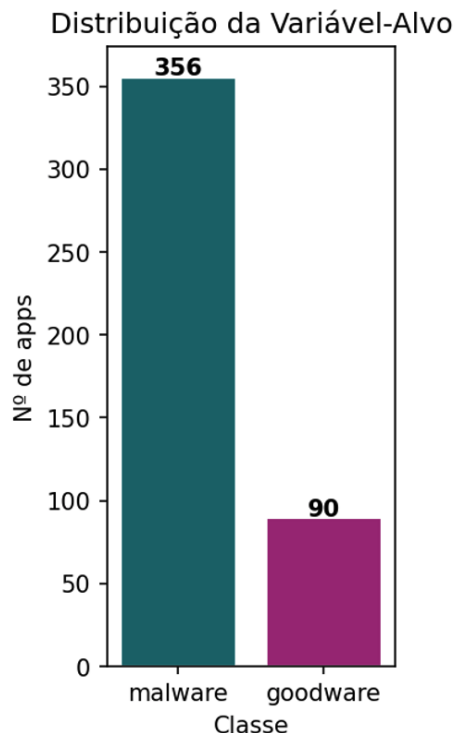


Figura 1: Gráfico de distribuição das classes (fonte: própria)

3.2 Distribuição Individual das Features de Interesse

Para além da variável-alvo, a exploração da subamostra examinou a distribuição individual das *features* independentes de maior relevância, separando os valores por classe (malware vs. goodware). Por se tratar de variáveis binárias, a distribuição de cada *feature* resume-se à proporção de instâncias que apresentam o valor 1 (presença) em cada classe.

A Figura 2 apresenta a taxa de presença (proporção de instâncias com valor 1) das cinco *features* mais discriminativas, estratificada por classe. O padrão observado revela dois comportamentos distintos. A *feature* `RECEIVE_BOOT_COMPLETED` domina claramente nas instâncias malware (98%) em comparação ao goodware (39%), refletindo o comportamento típico de malware persistente que se registra para execução automática no boot do dispositivo. As demais *features* — `openConnection`, `getLastKnownLocation`, `load` e `loadClass` — apresentam taxa de presença superior no goodware, o que é tecnicamente consistente: aplicativos legítimos utilizam explicitamente essas APIs para comunicação de rede e acesso à localização, enquanto malware tende a evitá-las em favor de canais menos rastreáveis. O poder discriminativo dessas *features* decorre, portanto, da sua **ausência** nas instâncias malware — o modelo aprende que `openConnection` = 0 é um indicador suspeito, não o contrário.

Essa assimetria entre classes confirma que as *features* selecionadas não apenas apresentam alta variância global — o que poderia decorrer de distribuição uniforme em

ambas as classes — mas concentram sua variância justamente na separação entre malware e goodware, propriedade essencial para a eficácia dos classificadores supervisionados.

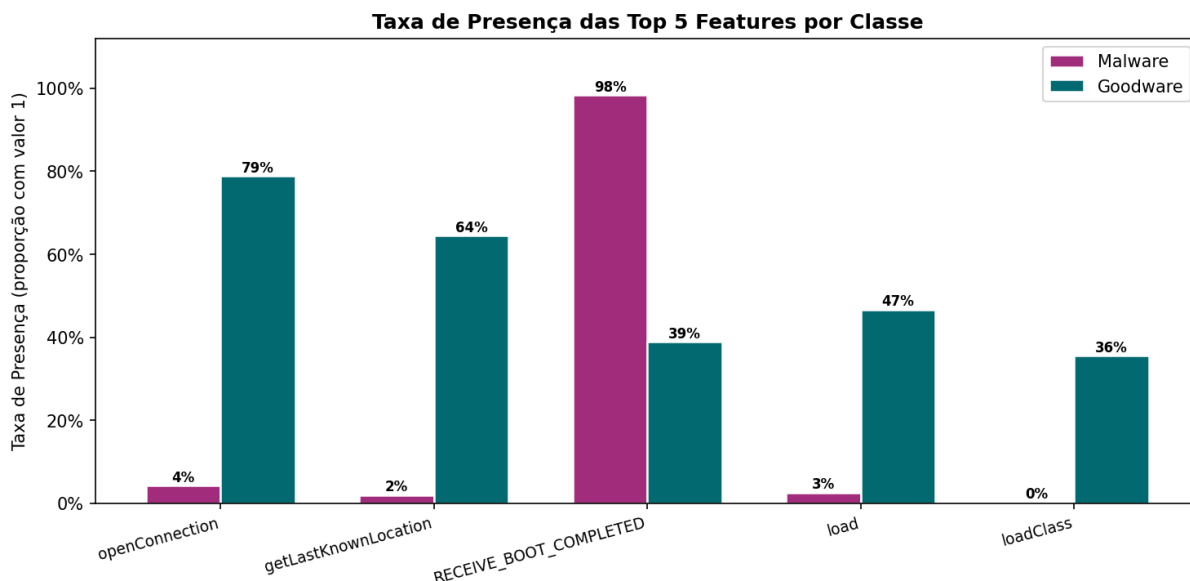


Figura 2: Taxa de presença das 5 *features* mais discriminativas, estratificada por classe (fonte: própria)

3.3 Features com Maior Poder Discriminativo

A análise de correlação entre as *features* e a variável-alvo, calculada sobre a subamostra, revelou cinco variáveis com maior poder discriminativo em valor absoluto (Tabela 2). As correlações negativas indicam associação com goodware — ou seja, a **ausência** dessas *features* é o sinal discriminativo para malware — enquanto a correlação positiva de `RECEIVE_BOOT_COMPLETED` indica presença predominante nas instâncias malware. Todas possuem interpretação técnica consistente com comportamentos conhecidos na literatura de segurança Android, validando a relevância do *dataset* para o objetivo da pesquisa.

Tabela 2: Top 5 *features* com maior correlação com a variável-alvo

Feature	Correlação com malware	Interpretação técnica
Ljava/net/URL;->openConnection	-0,760	Goodware usa mais — ausência é indicador de malware
LocationManager;->getLastKnownLocation	-0,711	Goodware usa mais — típico de <i>spyware</i> legítimo
RECEIVE_BOOT_COMPLETED	+0,694	Malware domina — execução persistente no boot
Ljava/lang/System;->load	-0,557	Goodware usa mais — carregamento explícito de bibliotecas
DexClassLoader;->loadClass	-0,541	Goodware usa mais — modularidade em apps legítimos

Nota: correlações negativas indicam associação com *goodware*; a tabela ordena por valor absoluto, pois ambos os sinais têm poder discriminativo equivalente.

A Figura 3 complementa a análise apresentada na Tabela 2, exibindo as 15 *features* com maior variância na subamostra (painel central) e as 15 *features* com maior correlação absoluta com a variável-alvo (painel direito). A proximidade entre os dois *rankings* confirma que as *features* mais variáveis são também as mais discriminativas, sugerindo que a variância constitui um indicador preliminar eficaz do poder preditivo em *datasets* de *features* binárias como o TUANDROMD. O painel esquerdo reforça o desbalanceamento entre as classes, já documentado na Seção 2.2.3, evidenciando a predominância de instâncias *malware* (79,8%) na subamostra exploratória.

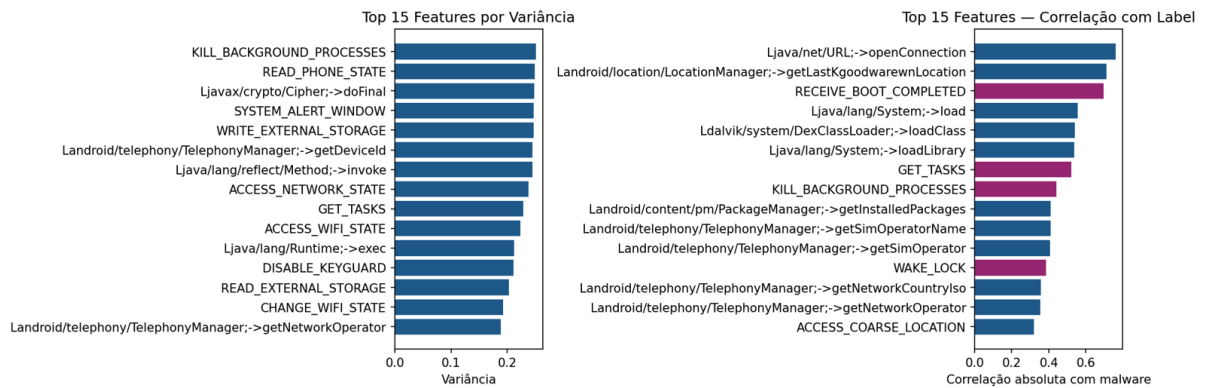


Figura 3: Top 15 *features* por variância e correlação absoluta com a variável-alvo (fonte: própria)

3.4 Questão de Pesquisa Refinada

A análise exploratória da subamostra permitiu refinar a questão de pesquisa inicialmente proposta. As cinco *features* de maior correlação absoluta com a variável-alvo pertencem a dois grupos funcionais bem definidos: APIs de conectividade e acesso a dados sensíveis

(`Ljava/net/URL;->openConnection`, correlação absoluta 0,760 — associação negativa, goodware usa mais, ausência é indicador de malware; `LocationManager;->getLastKnownLocation`, correlação absoluta 0,711 — associação negativa, goodware usa mais, ausência é indicador de malware) e mecanismos de execução dinâmica e persistência (`RECEIVE_BOOT_COMPLETED`, correlação 0,694 — associação positiva, malware domina; `Ljava/lang/System;->load`, correlação absoluta 0,557; `DexClassLoader;->loadClass`, correlação absoluta 0,541). Esses padrões são consistentes com comportamentos descritos na literatura como indicadores de malware Android sofisticado, como *spyware*, Trojans bancários e malware polimórfico.

Esses achados são corroborados por literatura recente: análise de importância de *features* com Random Forest sobre o TUANDROMD identificou `RECEIVE_BOOT_COMPLETED` e `Ljava/net/URL;->openConnection` como as principais *features* estáticas discriminativas entre malware e aplicações benignas (SAEED et al., 2026). A convergência entre os resultados da exploração e os achados da literatura reforça a validade metodológica da subamostra utilizada e a representatividade das *features* selecionadas para a fase confirmatória.

A partir desse resultado, a questão de pesquisa refinada passa a ser: Algoritmos de Aprendizado de Máquina Supervisionado treinados sobre *features* estáticas de permissões e chamadas a APIs Android — com ênfase na **ausência** de APIs de rede e na **presença** de mecanismos de persistência — são capazes de distinguir de forma estatisticamente significativa aplicativos maliciosos de benignos no *dataset* TUANDROMD, e qual algoritmo apresenta o melhor desempenho medido pelo *F1-score* sob validação cruzada estratificada?

A questão geral, enunciada na Seção 1.2, permanece como registro do percurso metodológico que conduziu a essa formulação mais precisa, conforme recomendado pelo ciclo exploratório-confirmatório adotado neste estudo.

4 PASSOS FUTUROS E CRONOGRAMA

A execução do estudo, a ser realizada no TP2, seguirá o cronograma semanal abaixo:

Tabela 3: Cronograma de execução do TP2

Semana	Período	Atividade
1	07–13/04	Pré-processamento: curadoria completa, normalização (SVM/KNN), <i>encoding</i> da variável-alvo
2	14–19/04	Entrega TP1 (19/04) + treinamento inicial: Random Forest e Árvore de Decisão com 10- <i>fold</i>
3	20–27/04	Treinamento: SVM e KNN; coleta e registro dos F1- <i>scores</i> por <i>fold</i>
4	28/04–05/05	Análise estatística: Wilcoxon pareado, IC, tabelas comparativas de desempenho
5	06–12/05	Redação TP2: introdução, métodos, resultados, discussão e referências

Os dados serão manipulados seguindo um *pipeline* reproduzível, executado integralmente no ambiente com Python 3.x e as bibliotecas `pandas`, `scikit-learn` e `numpy`. O *dataset* limpo de 4.464 instâncias será dividido em duas partições exclusivas: os 10% exploratórios já utilizados (446 instâncias, reservadas para esta fase) e os 90% restantes (4.018 instâncias), destinados exclusivamente à fase de treinamento e avaliação no TP2.

A curadoria dos dados já foi concluída na fase exploratória: a instância com rótulo ausente foi removida e nenhum valor faltante remanesceu após o procedimento. Para a fase de treinamento, será aplicada normalização Min-Max (escalamento para o intervalo $[0, 1]$) exclusivamente para os algoritmos sensíveis à escala — SVM e KNN —, enquanto Random Forest e Árvore de Decisão, por serem baseados em particionamento recursivo, dispensam esse pré-processamento. A normalização será ajustada (*fit*) apenas sobre os dados de treinamento de cada *fold* e aplicada (*transform*) sobre o *fold* de validação correspondente, impedindo qualquer vazamento de informação entre partições.

A avaliação dos modelos será realizada por meio de validação cruzada estratificada com $k = 10$ *folds* sobre as 4.018 instâncias reservadas, gerando 10 valores de F1-*score* por algoritmo. Esses valores alimentarão o teste de Wilcoxon pareado ($\alpha = 0,05$) para verificação de diferença estatisticamente significativa entre os pares de algoritmos (DEMŠAR, 2006). Todos os experimentos serão executados em ambiente local com Python 3.x, com `random_state=42` fixado em todos os passos estocásticos para garantir reprodutibilidade integral dos resultados.

Cabe ainda observar que a presença de 3.802 instâncias com vetores duplicados no *dataset* — mantidas por razões metodológicas detalhadas na Seção 2.2.2 — pode influenciar as estimativas de performance caso um mesmo vetor apareça simultaneamente em *folds* de treino e validação durante a validação cruzada. Esse efeito será monitorado no TP2 por meio da análise da variância dos F1-*scores* entre os 10 *folds*: alta variância entre folds pode

indicar instabilidade decorrente desse fenômeno, sinalizando a necessidade de reavaliação da decisão de manutenção das duplicatas.

5 REFERÊNCIAS

SHATNAWI, A. S. et al. **TUANDROMD: Tezpur University Android Malware Dataset**. UCI Machine Learning Repository, 2022. Disponível em: <https://archive.ics.uci.edu/dataset/855/tuandromd>. Acesso em: 05 abr. 2026.

SHATNAWI, A. S. et al. **TUNADROMD: Tezpur University Android Malware Dataset (versão pré-processada)**. UCI Machine Learning Repository, 2023. Disponível em: <https://archive.ics.uci.edu/dataset/813/tunadromd>. Acesso em: 05 abr. 2026.

ROMAN, N. T. **SIN5032 — Experimentação em Aprendizado de Máquina Supervisionado: Notas de Aula**. Universidade de São Paulo, EACH, 2026.

ALPAYDIN, E. **Introduction to Machine Learning**. 3. ed. Cambridge: MIT Press, 2014.

IZBICKI, R.; SANTOS, T. M. **Aprendizado de Máquina: uma abordagem estatística**. 2022. Disponível em: <http://www.rizbicki.ufscar.br/AME.pdf>. Acesso em: 05 abr. 2026.

AV-TEST INSTITUTE. **AV-ATLAS: Android Malware Statistics**. Disponível em: <https://www.av-test.org>. Acesso em: 05 abr. 2026.

KASPERSKY LAB. **The Mobile Threat Landscape in 2024**. Securelist, março de 2025. Disponível em: <https://securelist.com/mobile-threat-report-2024/115494>. Acesso em: 05 abr. 2026.

SAEED, N. H. et al. Efficient feature ranked hybrid framework for android IoT malware detection. **Scientific Reports**, v. 16, art. 3726, 27 jan. 2026. Disponível em: <https://www.nature.com/articles/s41598-026-35238-6>. Acesso em: 05 abr. 2026.

DEMŠAR, J. Statistical comparisons of classifiers over multiple data sets. **Journal of Machine Learning Research**, v. 7, p. 1–30, jan. 2006. Disponível em: <https://jmlr.csail.mit.edu/papers/volume7/demsar06a/demsar06a.pdf>. Acesso em: 05 abr. 2026.

ISAAC, J.; TANIMU, J. Android malware detection using random forest algorithm. **Zenodo**, mar. 2025. DOI: 10.5281/zenodo.14994363. Disponível em: <https://zenodo.org/records/14994363>. Acesso em: 05 abr. 2026.

VAROQUAUX, G. Cross-validation failure: Small sample sizes lead to large error bars. **NeuroImage**, v. 180, p. 68–77, 2017. Disponível em: <https://doi.org/10.1016/j>.

[neuroimage.2017.06.061](#). Acesso em: 15 abr. 2026.

SOUZA, T. et al. Outsmarting Android Malware with Cutting-Edge Feature Selection. **Computers, Materials & Continua**, v. 79, n. 1, 2024. Disponível em: <https://www.techscience.com/cmc/v79n1/56274>. Acesso em: 15 abr. 2026.

SOUZA, J. V. S. et al. Imbalanced malware classification: an approach based on dynamic classifier selection. **arXiv**, 2504.00041, abr. 2025. DOI: [10.48550/arXiv.2504.00041](https://arxiv.org/abs/2504.00041). Disponível em: <https://arxiv.org/abs/2504.00041>. Acesso em: 05 abr. 2026.